Word Vector Representations Word2Vec

Dr.Srijith P.K.
Dept of CSE and Dept of AI
Indian Institute of Technology Hyderabad

Analogical Reasoning

- Analogical reasoning plays in human cognition for discovering new knowledge and understanding new concepts

Athens:Greece:: Oslo: ?

Switzerland : Swiss :: Cambodia : ?

USA : Dollar :: India : ?

Einstein : scientist :: Messi: ?

One hot vector representation

	Dog	Cat	Python
Document 1	1	1	0
Document 2	1	0	0
Document 3	0	0	1

- Dog = [0,0,1,0,…,0]
- Cat = [0,1,0,…,0]
- Python = [1,0,…,0]
- Does not capture semantic similarity

"The cat is walking in the bedroom"

"A dog was running in a room"

Vector Semantics and Embeddings

- Vector semantics : Represent words in a lower-dimensional space where it can capture semantics of the word. Vectors representing words are called embeddings (often dense vector representations).

- Distributional Hypothesis : Words that occur in similar contexts tend to have similar meanings.

Word	w	$C(w)$
"the"	1	[0.6762, -0.9607, 0.3626, -0.2410, 0.6636]
"a"	2	[0.6859, -0.9266, 0.3777, -0.2140, 0.6711]
"have"	3	[0.1656, -0.1530, 0.0310, -0.3321, -0.1342]
"be"	4	[0.1760, -0.1340, 0.0702, -0.2981, -0.1111]
"cat"	5	[0.5896, 0.9137, 0.0452, 0.7603, -0.6541]
"dog"	6	[0.5965, 0.9143, 0.0899, 0.7702, -0.6392]
"car"	7	[-0.0069, 0.7995, 0.6433, 0.2898, 0.6359]

Distributional Hypothesis

Distributional hypothesis. first formulated in the 1950s by linguists like Joos (1950) and Firth (1957).

- Two words that occur in very similar distributions (whose neighboring words are similar) have similar meanings.

"You shall know a word by the company it keeps!" - John Firth

Whats the meaning of the word "ongchoi" ?

Joos, M. (1950). Description of language design. JASA 22, 701–708.
Firth, J. R. (1957). A synopsis of linguistic theory 1930– 1955. Studies in Linguistic Analysis. Philological Society. Reprinted in Palmer, F. (ed.) 1968. Selected Papers of J. R. Firth. Longman, Harlow.

Distributional Hypothesis

- Two words that occur in very similar distributions (whose neighbouring words are similar) have similar meanings.

Whats the meaning of the word “ongchoi” ?

Ongchoi is delicious sauteed with garlic.
Ongchoi is superb over rice.
...ongchoi leaves with salty sauces…

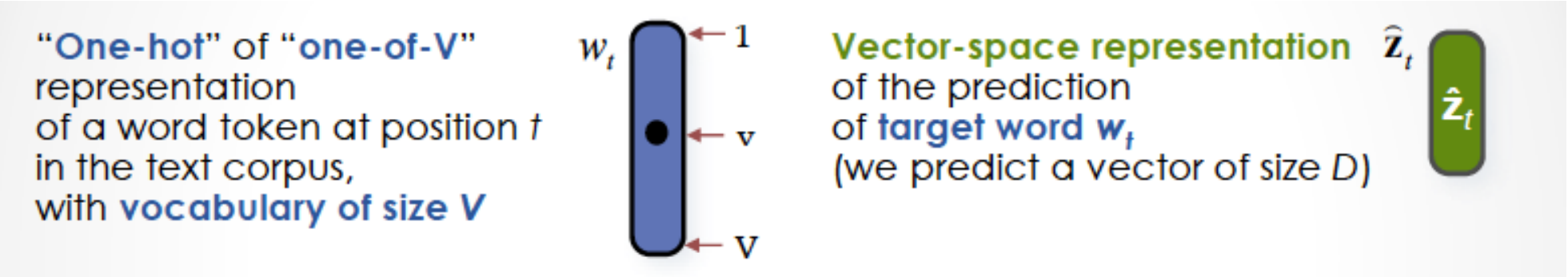
...spinach sauteed with garlic over rice…
...chard stems and leaves are delicious…
...collard greens and other salty leafy greens

Vector semantics

Vectors-space representation based on distributional hypothesis enables capturing syntactic/semantic similarity between words

- Find nearest neighbours : synonyms , antonyms
- Algebra on words : analogical reasoning
- Helps in better in better generalisation performance for NLP applications.

- Count based (using co-occurrence frequencies as in LSA, LDA) vs
- Predictive models (NLM, word2vec, Glove, BERT etc.)



Predictive Approaches

- Learn word embeddings using a predictive (machine learning) model.
- Input : word/sequence of words
- Output : target word
- Learn a function (machine learning model) which can model the probability of predicting the target word given input word/words

$$p(w_t \mid w_{t-1}, w_{t+1}) = f(w_{t-1}, w_{t+1})$$

[tablespoon of apricot jam]

p(apricot | of, jam) = f(of, jam)

Predictive Approaches

- Learn word embeddings using a predictive (machine learning) model.
- Input : word/sequence of words
- Output : target word
- Learn a function (machine learning model) which can model the probability of predicting the target word given input word/words

$$p(w_t \mid w_{t-1}, w_{t+1}) = f(w_{t-1}, w_{t+1})$$ Continuous BOW

$$p(w_{t-1} \mid w_t) = f(w_t)$$ Skip-gram

$$p(w_t \mid w_{t-1}, w_{t-2}) = f(w_{t-1}, w_{t-2})$$ Neural LM

Predictive Approaches

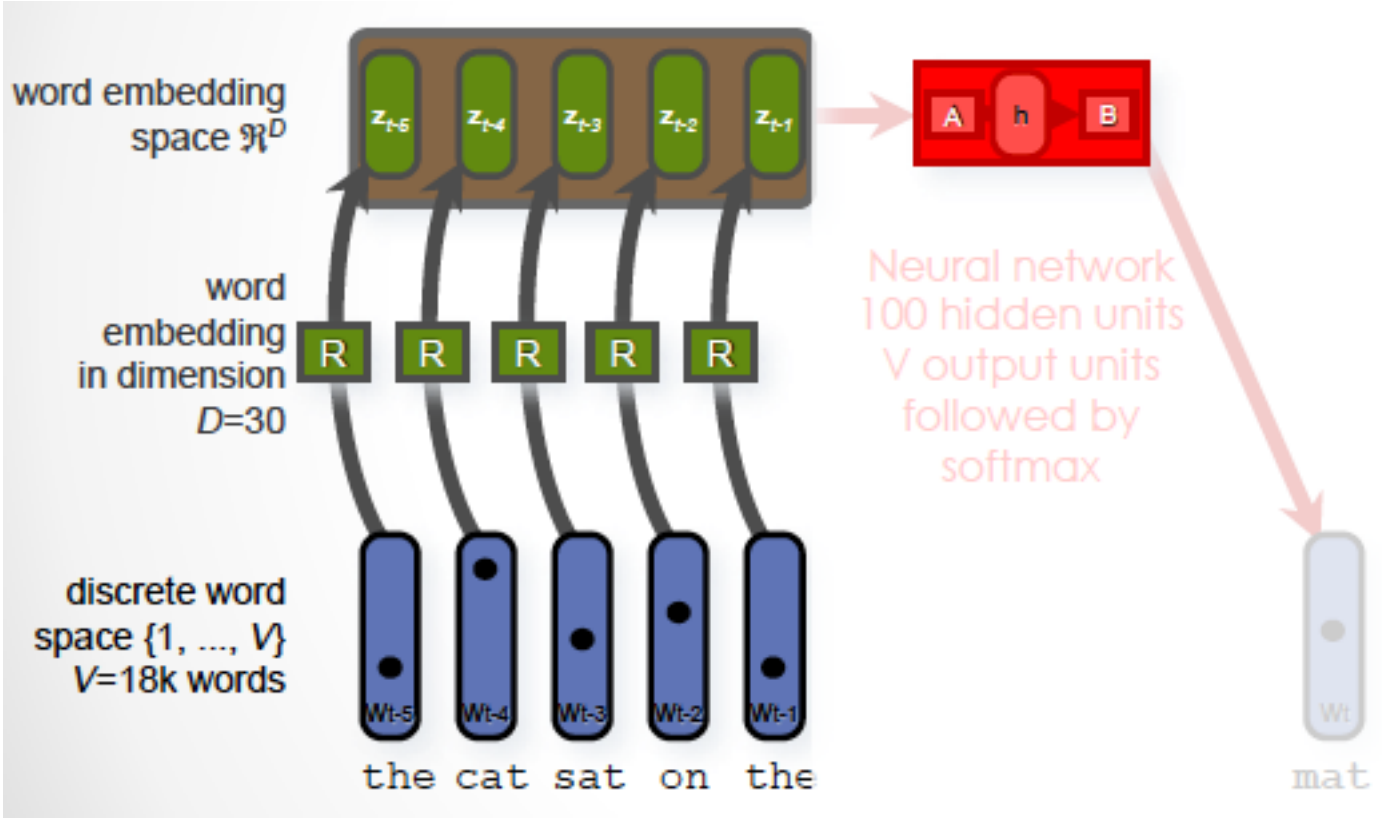
Allows us to learn word embeddings with semantic interpretation

- short dense real valued vectors, size typically vary from 50-1000
- Improves generalisation performance for machine learning algorithms by capturing semantic relatedness, reduced number of parameters through dense representation

- Allows self-supervised learning of embedding using text (sequence of words) in documents as implicitly supervised training data
- Pre-trained word embeddings can be used to improve performance of several NLP applications like machine translations, question answering , information extraction.

Predictive Approaches

- Bengio et al. (2003) learnt embeddings of words for neural language model, where they use a neural network to predict next word in a sequence using past words



Yoshua Bengio, Rejean Ducharme, Pascal Vincent, and Christian Jauvin. 2003. A neural probabilistic language model. In *Journal of Machine Learning Research*, pages 1137–1155.

Word2vec

- Word embeddings learnt from NPLM are more suitable for language modelling task
- Word2vec provides large improvements in accuracy at much lower computational cost due to a simpler model (log-linear classifier).
- Word2vec Tool : consists of continuous bag of words (CBOW) and Skip-gram models to capture semantic relatedness in words efficiently and effectively.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient Estimation of Word Representations in Vector Space. In Proceedings of Workshop at ICLR, 2013.
Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed Representations of Words and Phrases and their Compositionality. In Proceedings of NIPS, 2013.

https://code.google.com/archive/p/word2vec/

Word2vec

- CBOW predicts target words (e.g. ‘sits') from source context('the cat sits on the'), while the skip-gram does the inverse and predicts source context-words from the target words.

CBOW

Skip-gram

$$p(w_t \mid w_{t-1}, w_{t+1}) = f(w_{t-1}, w_{t+1}) \quad ; \quad p(w_{t-1} \mid w_t) = f(w_t)$$

the	cat	sits	on	the	mat
C1	C2	Target	C3	C4	

p(sits|the,cat,on,the)
= f(the,cat,on,the)

p(cat|sits) = f(sits)
p(on|sits) = f(sits)

Word2vec

- CBOW predicts target words (e.g. 'mat') from source context('the cat sits on the'), while the skip-gram does the inverse and predicts source context-words from the target words.

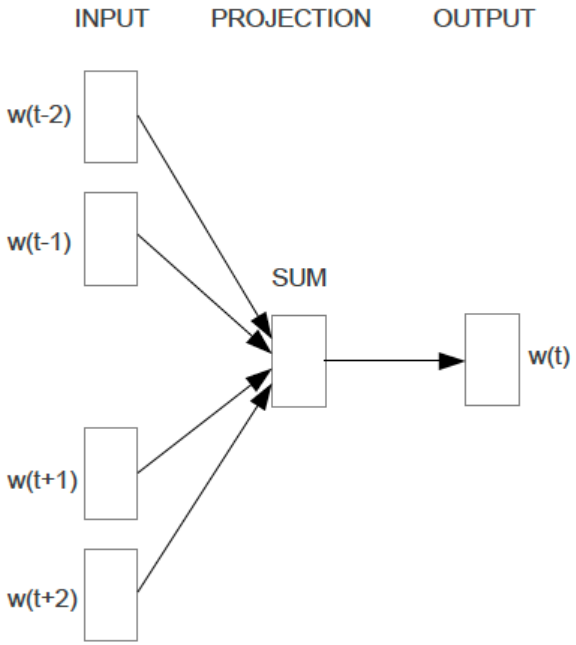
- CBOW treats entire context as one observation (useful for smaller datasets) while skip-gram treats each context-target pair as a new observation (useful for larger datasets)

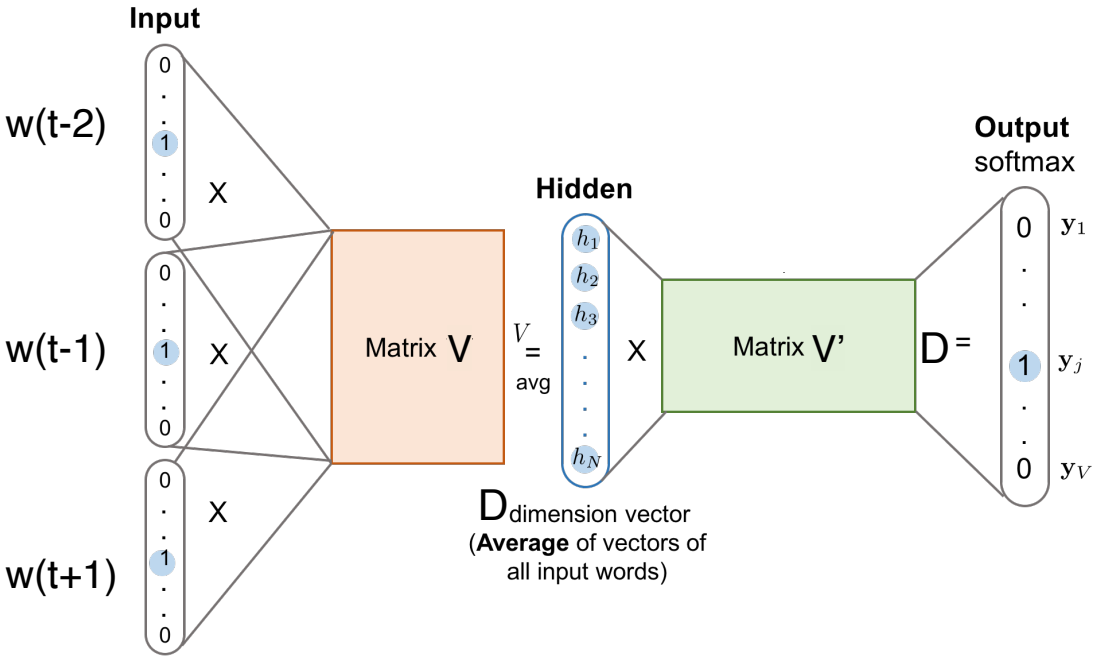
the quick brown fox jumped over the lazy dog

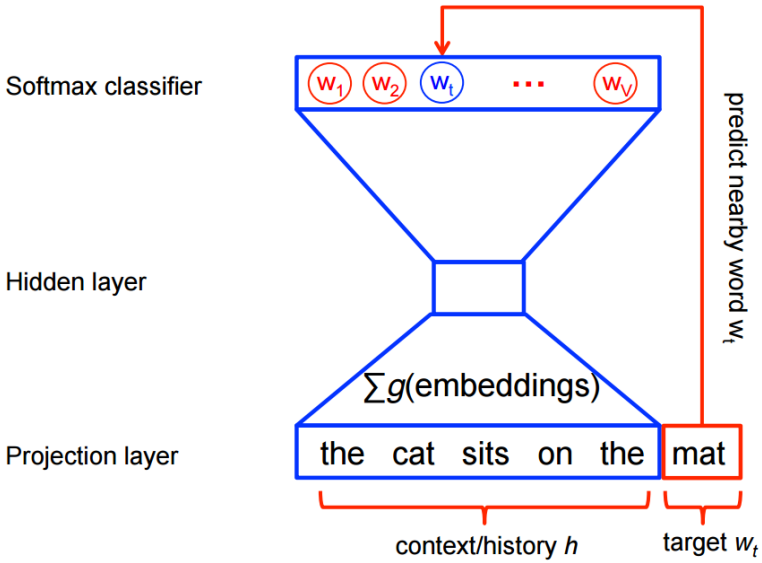
CBOW : ([the, brown], quick), ([quick, fox], brown), ([brown, jumped], fox),…

Skip-gram : (quick, the), (quick, brown), (brown, quick), (brown, fox), …

Continuous bag of words Word2vec





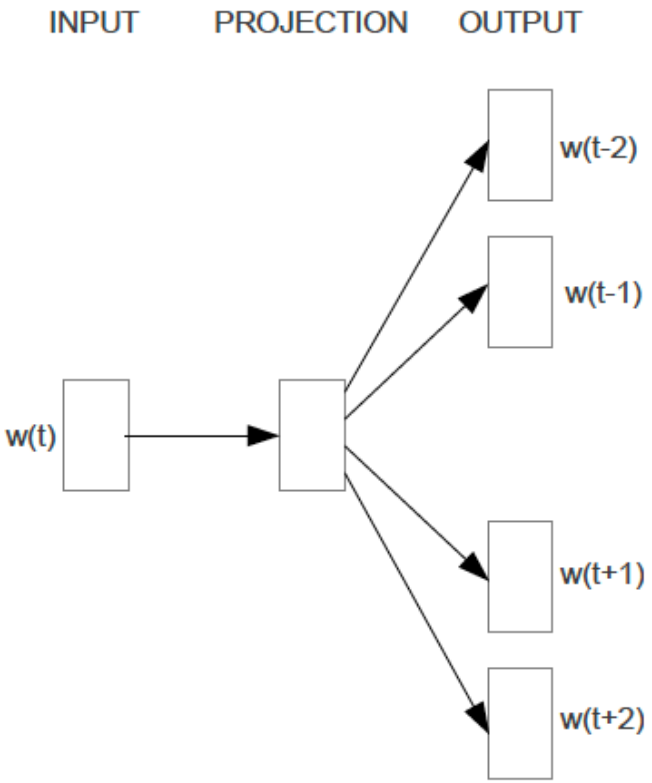


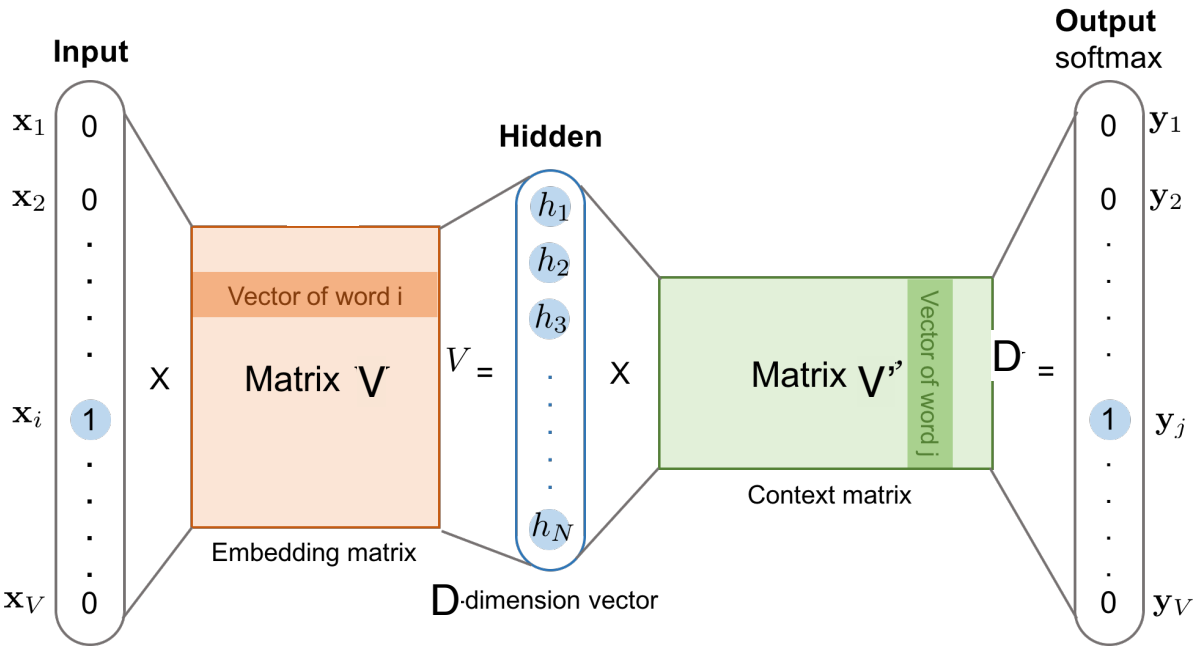
$$p(w_O|w_I) = \frac{\exp\left({v'_{w_O}}^{\top} v_{w_I}\right)}{\sum_{w=1}^{W} \exp\left({v'_w}^{\top} v_{w_I}\right)}$$

$$J_\theta = \frac{1}{T}\sum_{t=1}^{T} \log p(w_t \mid w_{t-n}, \cdots, w_{t-1}, w_{t+1}, \cdots, w_{t+n}).$$

https://lilianweng.github.io/lil-log/2017/10/15/learning-word-embedding.html

Skip-gram Word2vec





$$p(w_O|w_I) = \frac{\exp\left({v'_{w_O}}^\top v_{w_I}\right)}{\sum_{w=1}^{W} \exp\left({v'_w}^\top v_{w_I}\right)}$$

$$J_\theta = \frac{1}{T}\sum_{t=1}^{T} \sum_{-c \le j \le c, j \neq 0} \log p(w_{t+j}|w_t)$$

https://lilianweng.github.io/lil-log/2017/10/15/learning-word-embedding.html

Word2vec training

- Trained by maximising log-likelihood over data
- Skipgram objective function

$$\sum_{(w_O,w_I)\in D} \log p(w_O \mid w_I) = \sum_{(w_O,w_I)\in D} \left(\log \exp({v'_{w_O}}^{\top} v_{w_I}) - \log \sum_{w=1}^{W} \exp({v'_w}^{\top} v_{w_I})\right)$$

- Softmax evaluation is computationally costly
- Noise Contrastive Estimation (NCE) metric intends to differentiate the target word from noise samples using a logistic regression classifier.
- Softmax approximated by logistic, p(+1|x) = $\sigma(x) = \frac{1}{1+e^{-x}}$

$$\log \sigma({v'_{w_O}}^{\top} v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-{v'_{w_i}}^{\top} v_{w_I})\right]$$

$p(+1 \mid w_O, w_I)$ $\qquad$ $p(-1 \mid w_i, w_I)$

Skip-gram training

1. Treat the target word t and a neighboring context word c as positive examples.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

Assume a +/- 2 word window, given training sentence:

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 c3 c4

Dan Jurafsky and James H. Martin, Speech and Language Processing (3rd ed. draft), 2020

Skip-gram training

Assume a +/- 2 word window, given training sentence:

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 Target c3 c4

Goal: train a classifier that is given a candidate (word, context) pair

(apricot, jam) p(+1|apricot, jam) $= \sigma((v'_{w_{jam}}{}^{\top} v_{w_{apricot}}))$

(apricot, aardvark) p(-1| apricot, aardvark) $= \sigma(-(v'_{w_{aardvark}}{}^{\top} v_{w_{apricot}}))$

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

$$P(-|w,c) = 1 - P(+|w,c)$$

$$= \sigma(-c \cdot w) = \frac{1}{1+\exp(c \cdot w)}$$

Skip-gram training data

…lemon, a [tablespoon of apricot jam, a] pinch…
c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

For each positive example, sample k negative examples from vocabulary (Negative sampling)

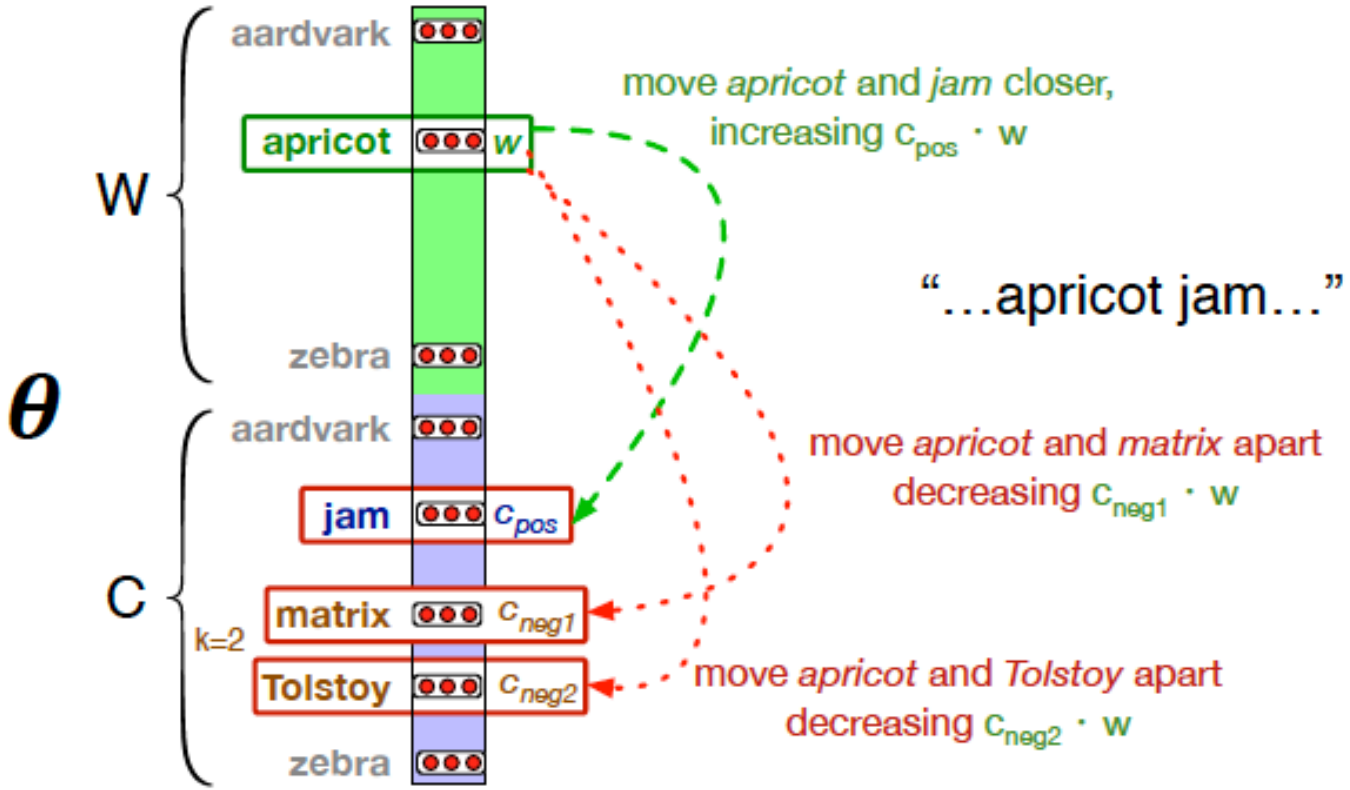
Skip-gram Loss derivation

Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the k negative sampled non-neighbor words.

$$
\begin{aligned}
L_{CE} &= -\log\left[P(+|w,c_{pos})\prod_{i=1}^{k}P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log\left(1-P(+|w,c_{neg_i})\right)\right] \\
&= -\left[\log \sigma(c_{pos}\cdot w) + \sum_{i=1}^{k}\log\sigma(-c_{neg_i}\cdot w)\right]
\end{aligned}
$$

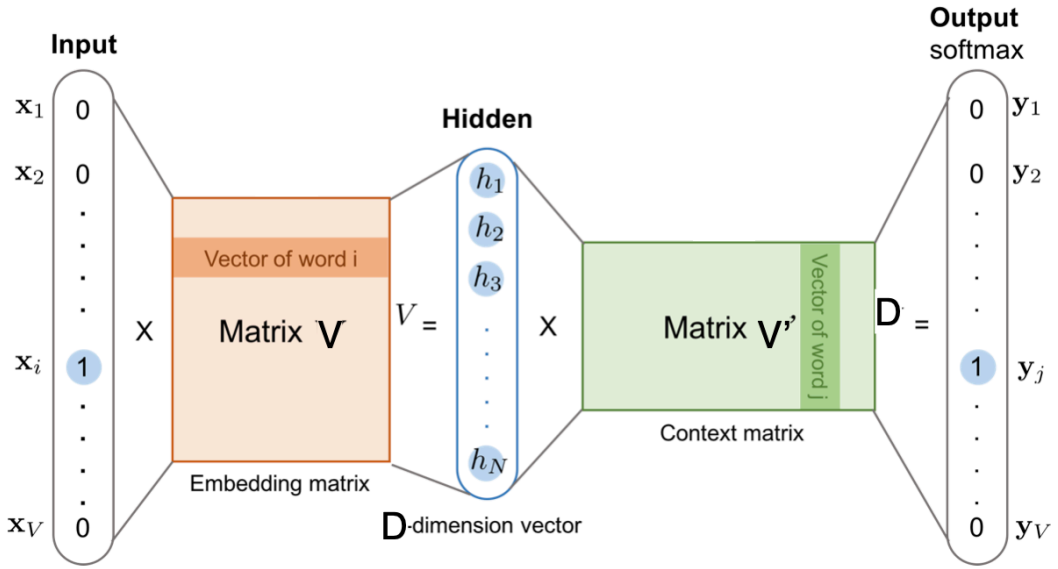
Skip-gram learning

Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the k negative sampled non-neighbor words.



Skip-gram training

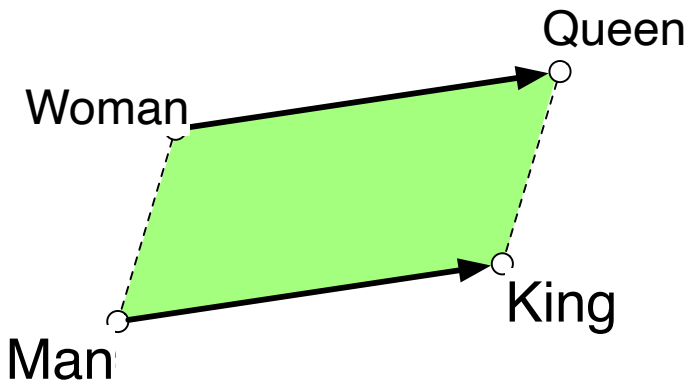
- Stochastic gradient descent on the loss to learn embeddings
- Skip-gram learns embeddings by starting with random embedding vectors of dimension d.
- Iteratively shifting the embedding of each word w to be more like the embeddings of words that occur nearby in texts, and less like the embeddings of words that don't occur nearby.
- Learns embedding matrix V and context matrix V', embedding of word I, is taken as V(i)+V'(i)

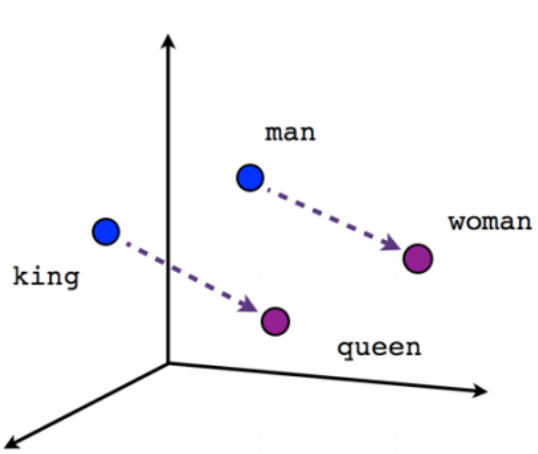


Word Embeddings with Word2vec

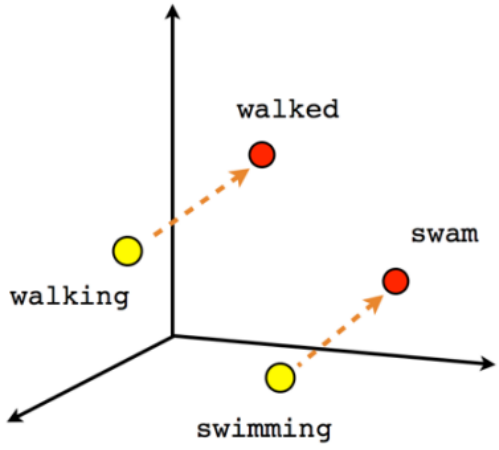
- Model analogical relations and can be useful for Analogical reasoning
- Man:King::women:?
- Parallelogram approach :

vec(King) - vec(Man) + vec(women) is close to vec(Queen)

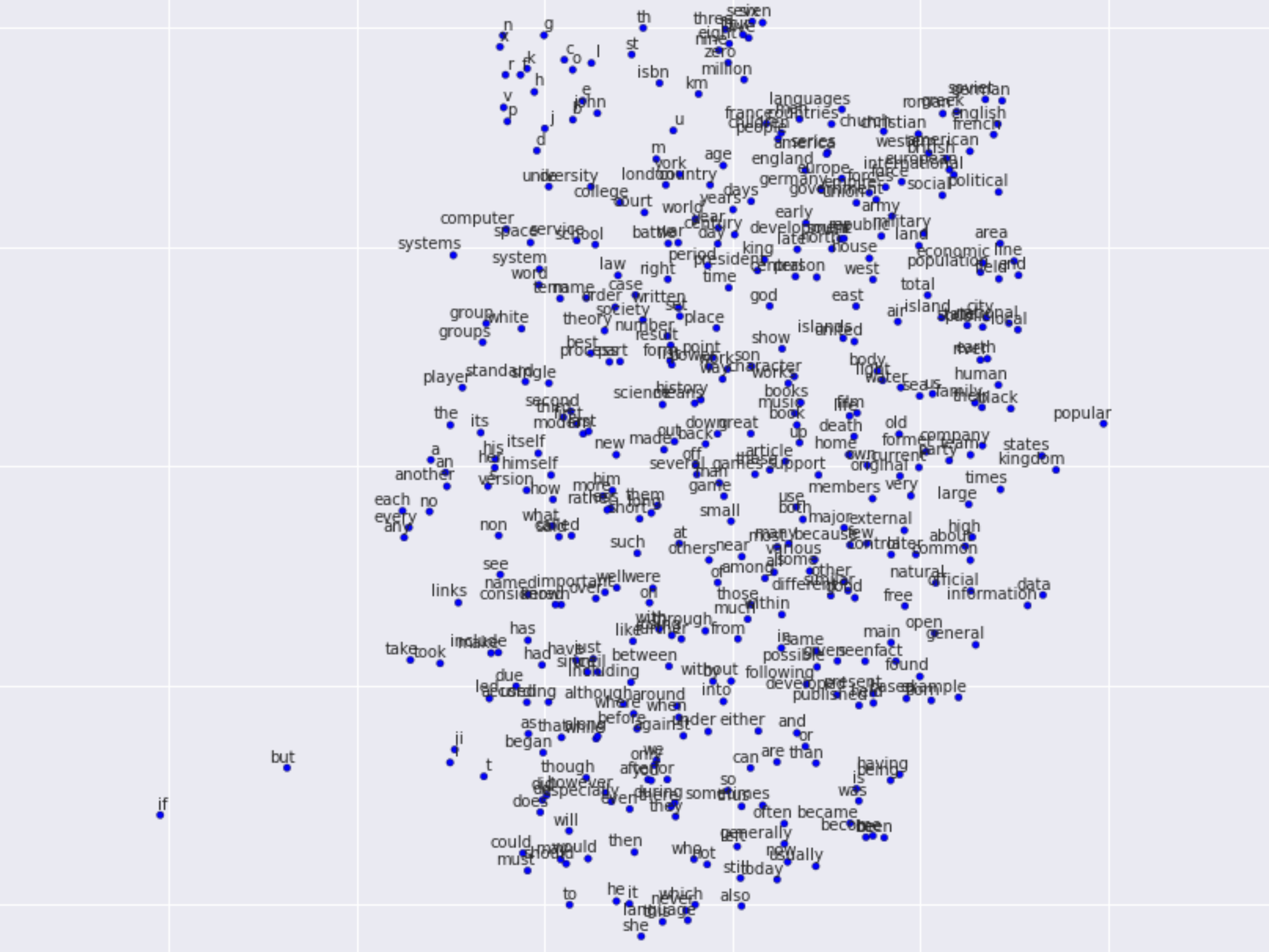




Male-Female



Verb tense


n
g
th
x
c
o
l
st
r
k
h
isbn
km
million
zero
nine
v
e
john
p
j
d
u
m
languages
countries
christian
english
french
american
age
england
european
international
political
social
university
college
court
days
years
world
army
early
military
computer
service
school
systems
area
king
house
system
law
right
period
time
word
name
case
west
total
population
economic
order
written
god
east
group
white
theory
society
place
groups
number
result
show
islands
united
best
point
son
character
works
body
player
standard
single
science
history
books
human
water
family
black
popular
second
music
film
life
book
the
its
modern
great
death
old
company
a
his
itself
new
made
out
back
up
home
former
states
an
himself
article
support
original
kingdom
another
version
several
games
times
game
members
very
large
him
more
them
each
no
how
rather
use
every
short
both
small
major
external
any
non
what
such
at
many
because
few
about
high
others
near
various
common
see
among
other
natural
named
important
well
were
different
official
data
links
those
free
information
within
much
main
open
through
from
general
has
like
take
took
had
just
between
possible
fact
found
include
have
without
following
due
example
although
around
into
published
where
when
as
before
against
either
and
or
ii
began
are
than
but
t
though
can
having
after
being
however
so
is
does
during
sometimes
was
especially
they
often
became
will
become
been
if
could
then
generally
who
not
now
must
usually
still
today
to
he
it
which
also
never
language
she

Word2vec(Tensorflow)

```
Srijiths-MacBook-Pro:LM srijith$ python word2vec_basic.py
Found and verified text8.zip
Data size 17005207
Most common words (+UNK) [['UNK', 418391], ('the', 1061396), ('of', 593677), ('and', 416629), ('one', 411764)
Sample data [5239, 3084, 12, 6, 195, 2, 3137, 46, 59, 156] ['anarchism', 'originated', 'as', 'a', 'term', 'of
 'against']
3084 originated -> 12 as
3084 originated -> 5239 anarchism
12 as -> 3084 originated
12 as -> 6 a
6 a -> 12 as
6 a -> 195 term
195 term -> 6 a
195 term -> 2 of

Average loss at step  28000 :  6.32716879857
Average loss at step  30000 :  5.93475983298
Nearest to called: destroyer, busts, ducas, agave, protected, second, variously, slated,
Nearest to up: mandaeans, win, vaccines, researchers, carnivorous, honorius, durability, co
Nearest to system: panoramic, agouti, operatorname, karloff, amaranthus, circ, flora, danie
Nearest to were: are, was, is, abet, be, deeply, have, address,
Nearest to its: their, the, his, gollancz, agatha, dasyprocta, him, operatorname,
Nearest to use: lnot, squirrel, draw, graves, abet, entertainment, abakan, juridical,
```

Experiment Results

- Overall, there are 8869 semantic and 10675 syntactic questions
- Google News corpus for training the word vectors, 6B tokens and 1M vocabulary size

Type of relationship	Word Pair 1		Word Pair 2	
Common capital city	Athens	Greece	Oslo	Norway
All capital cities	Astana	Kazakhstan	Harare	Zimbabwe
Currency	Angola	kwanza	Iran	rial
City-in-state	Chicago	Illinois	Stockton	California
Man-Woman	brother	sister	grandson	granddaughter
Adjective to adverb	apparent	apparently	rapid	rapidly
Opposite	possibly	impossibly	ethical	unethical
Comparative	great	greater	tough	tougher
Superlative	easy	easiest	lucky	luckiest
Present Participle	think	thinking	read	reading
Nationality adjective	Switzerland	Swiss	Cambodia	Cambodian
Past tense	walking	walked	swimming	swam
Plural nouns	mouse	mice	dollar	dollars
Plural verbs	work	works	speak	speaks

Experimental Results

The results using the CBOW architecture with different choice of word vector dimensionality and increasing amount of the training data

Dimensionality / Training words	24M	49M	98M	196M	391M	783M
50	13.4	15.7	18.6	19.1	22.5	23.2
100	19.4	23.1	27.8	28.7	33.4	32.2
300	23.2	29.2	35.3	38.6	43.7	45.9
600	24.0	30.1	36.5	40.8	46.6	50.4

Experimental Results

Comparison of architectures using models trained on the same data, with 640-dimensional word vectors. The training data consists of several LDC corpora (320M words, 82K vocabulary).

Model Architecture	Semantic-Syntactic Word Relationship test set	
	Semantic Accuracy [%]	Syntactic Accuracy [%]
RNNLM	9	36
NNLM	23	53
CBOW	24	64
Skip-gram	55	59

Model	Vector Dimensionality	Training words	Accuracy [%]			Training time [days x CPU cores]
			Semantic	Syntactic	Total	
NNLM	100	6B	34.2	64.5	50.8	14 x 180
CBOW	1000	6B	57.3	68.9	63.7	2 x 140
Skip-gram	1000	6B	66.1	65.1	65.6	2.5 x 125

Skip-gram relationships

Skipgram model trained on 783M words with 300 dimensionality

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Conclusion

Word2vec provided word embeddings (low-dimensional, dense representations of words) with a lower computational cost (using simple log-linear classifier) and improved generalisation performance on NLP tasks.

Skip-gram: works well with small amount of the training data, represents well even rare words or phrases.

CBOW: several times faster to train than the skip-gram, slightly better accuracy for the frequent words

Word2vec was found to be useful for many downstream tasks such as machine translation, sentiment analysis, named entity recognition.

word2vec methods are fast, efficient to train, and easily available online with code and pretrained embeddings. Use a pre-trained word2vec embedding on your natural language processing task.

Word2vec ideas has been extended for getting representation at subword level (**fasttext :** useful for dealing with out-of-vocabulary words) and at paragraph and document level.

Word2vec is a static embedding and is not contextual - words can exhibit different meaning in different contexts but word2vec learns a single representation for a word.

GloVe (Pennington et al., 2014) : Global Vectors capturing global corpus statistics.

Reference

Yoshua Bengio, Rejean Ducharme, Pascal Vincent, and Christian Jauvin. 2003. A neural probabilistic language model. In *Journal of Machine Learning Research*, pages 1137–1155.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient Estimation of Word Representations in Vector Space. In Proceedings of Workshop at ICLR, 2013.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed Representations of Words and Phrases and their Compositionality. In Proceedings of NIPS, 2013.

Bojanowski, P., Grave, E., Joulin, A., and Mikolov, T. (2017). Enriching word vectors with subword information. TACL 5, 135–146.

Pennington, J., Socher, R., and Manning, C. D. (2014). Glove: Global vectors for word representation. EMNLP.

Dan Jurafsky and James H. Martin, Speech and Language Processing (3rd ed. draft), 2020

Codes and Demos

https://code.google.com/archive/p/word2vec/

https://radimrehurek.com/gensim/models/word2vec.html

https://www.tensorflow.org/tutorials/text/word2vec

http://bionlp-www.utu.fi/wv_demo/

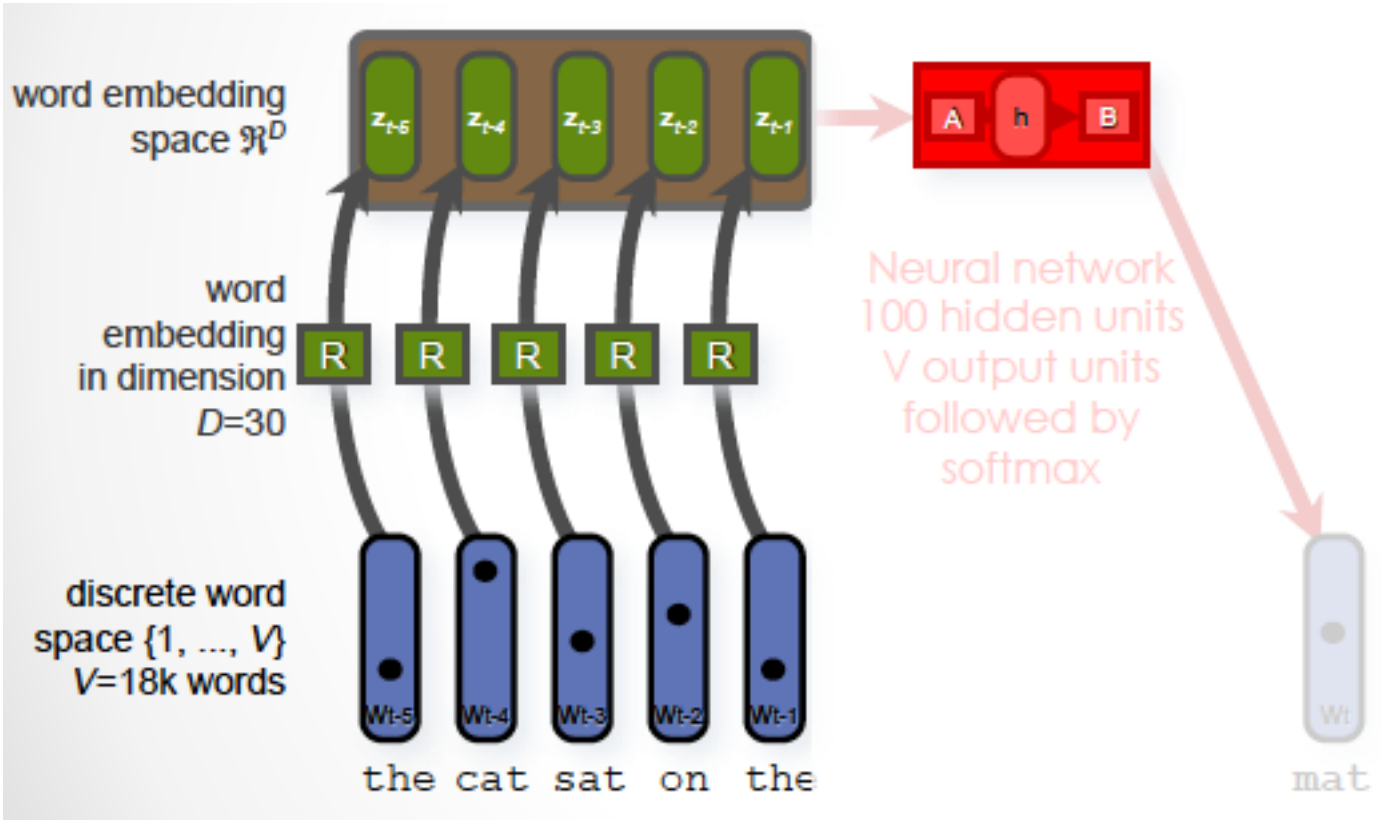
Neural Probabilistic Language Model

- Predictive approach to learn VSM

  Vector weights are learnt to predict the contexts in which the corresponding words tend to appear. Similar vectors to words occurring in similar contexts

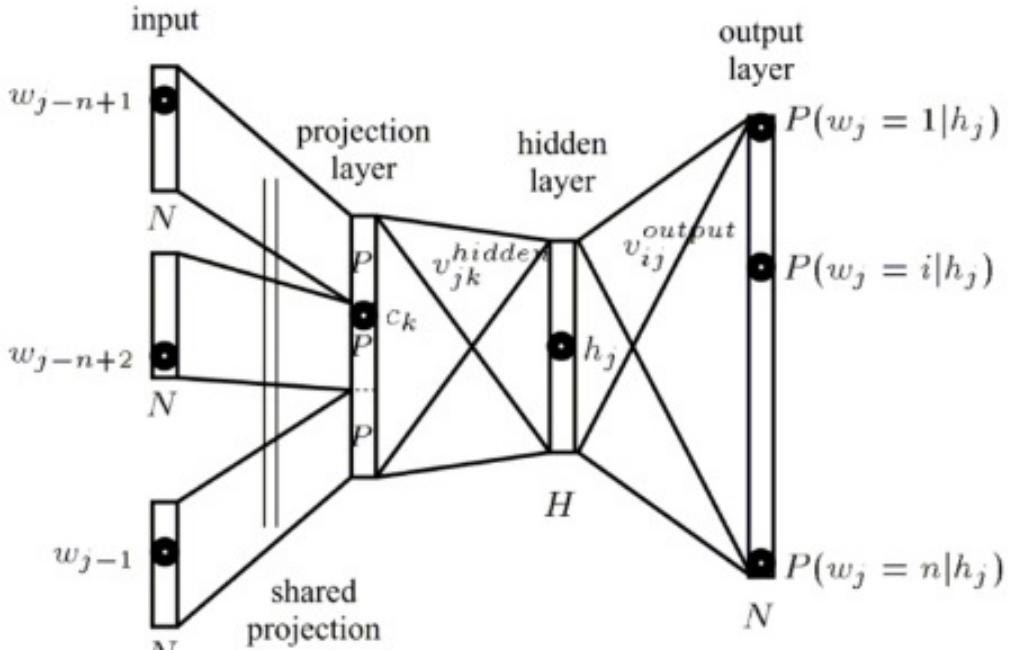
- Learn word embeddings which helps in improving the language modelling task
- Typical N-gram models
  - Suffers from data sparsity issues
  - Does not consider similarity between words and sentences
    - The Cat is walking in the bedroom
    - A Dog is running in the room

Neural probabilistic language model



Neural Probabilistic Language Model

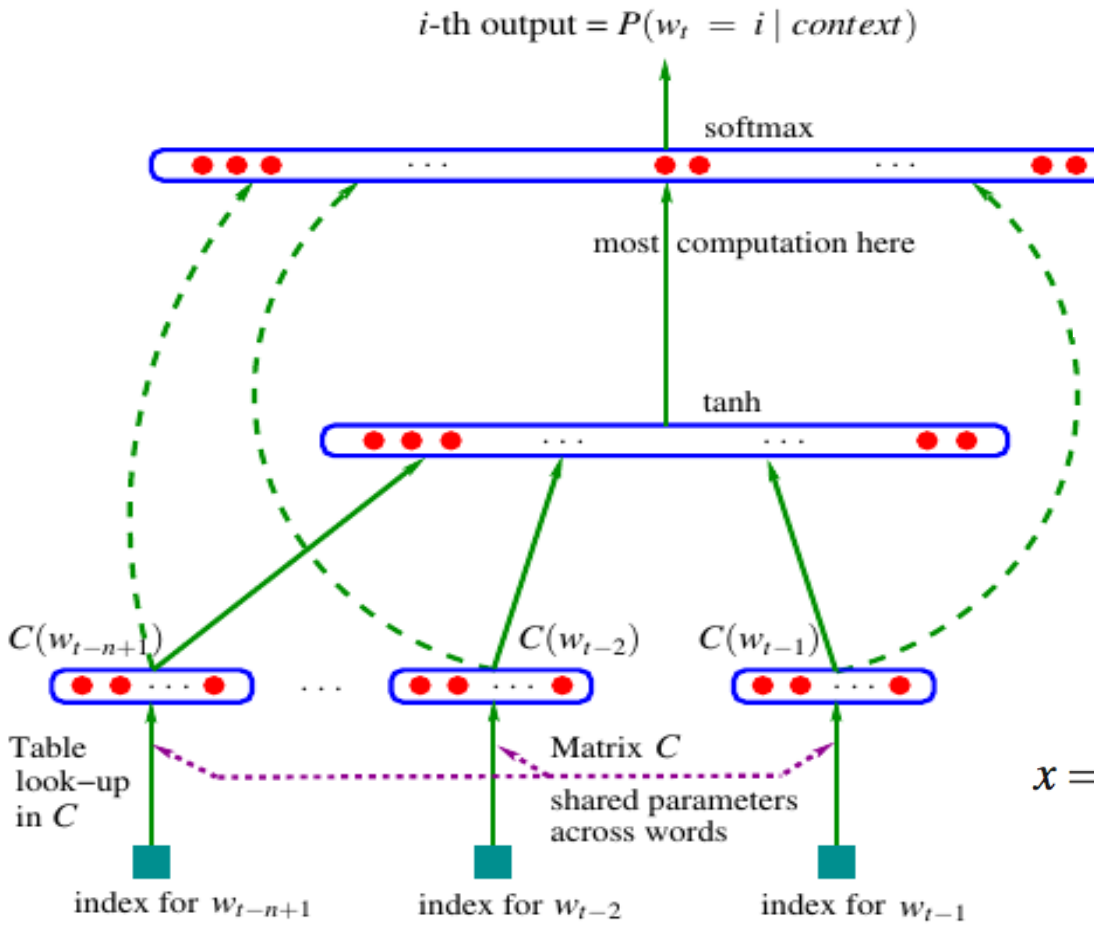
- Associate with each word in the vocabulary a distributed word feature vector (a real-valued vector in R^m) m << V
- Express the joint probability function of word sequences in terms of the feature vectors of these words in the sequence,
- Learn simultaneously the word feature vectors and the parameters of that probability function.
- The cat is walking in the bedroom ; A dog was running in a room
- The cat is running in a room ; A dog is walking in a bedroom



$$\hat{P}(w_1^T) = \prod_{t=1}^{T} \hat{P}(w_t | w_1^{t-1}),$$

$$p(w_t \mid w_{t-1}, \cdots, w_{t-n+1}) = \frac{\exp(h^\top v'_{w_t})}{\sum_{w_i \in V} \exp(h^\top v'_{w_i})}.$$

Neural Probabilistic Language Model



$$\hat{P}(w_t|w_{t-1},\cdots w_{t-n+1}) = \frac{e^{y_{w_t}}}{\sum_i e^{y_i}}.$$

$$y = b + Wx + U\tanh(d + Hx)$$

$$x = (C(w_{t-1}), C(w_{t-2}), \cdots, C(w_{t-n+1})).$$

$$f(w_t,\cdots,w_{t-n+1}) = \hat{P}(w_t|w_1^{t-1}) \qquad f(i,w_{t-1},\cdots,w_{t-n+1}) = g(i,C(w_{t-1}),\cdots,C(w_{t-n+1}))$$

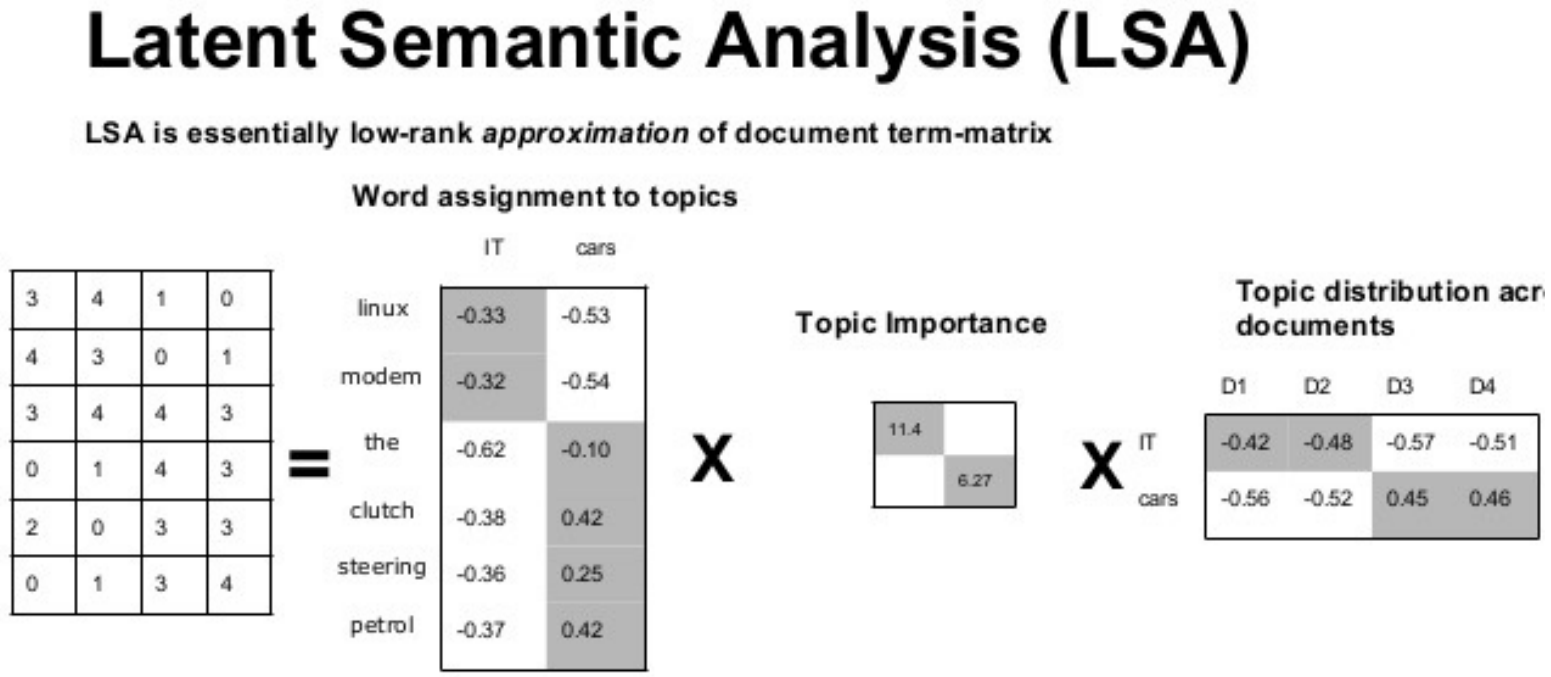
$$L = \frac{1}{T}\sum_t \log f(w_t,w_{t-1},\cdots,w_{t-n+1};\theta) + R(\theta), \quad |V|(1+nm+h) + h(1+(n-1)m).$$

Neural Probabilistic Language Model

	n	c	h	m	direct	mix	train.	valid.	test.
MLP1	5		50	60	yes	no	182	284	268
MLP2	5		50	60	yes	yes		275	257
MLP3	5		0	60	yes	no	201	327	310
MLP4	5		0	60	yes	yes		286	272
MLP5	5		50	30	yes	no	209	296	279
MLP6	5		50	30	yes	yes		273	259
MLP7	3		50	30	yes	no	210	309	293
MLP8	3		50	30	yes	yes		284	270
MLP9	5		100	30	no	no	175	280	276
MLP10	5		100	30	no	yes		265	**252**
Del. Int.	3						31	352	336
Kneser-Ney back-off	3							334	323
Kneser-Ney back-off	4							332	321
Kneser-Ney back-off	5							332	321
class-based back-off	3	150						348	334
class-based back-off	3	200						354	340
class-based back-off	3	500						326	**312**
class-based back-off	3	1000						335	319
class-based back-off	3	2000						343	326
class-based back-off	4	500						327	312
class-based back-off	5	500						327	312

LSA

- Count based (co-occurrence frequencies) approach to obtain Vector space representation
- Captures topical similarity



Predictive vs. count based

name	task
rg	relatedness
ws	relatedness
wss	relatedness
wsr	relatedness
men	relatedness
toefl	synonyms
ap	categorization
esslli	categorization
battig	categorization
up	sel pref
mcrae	sel pref
an	analogy
ansyn	analogy
ansem	analogy

	rg	ws	wss	wsr	men	toefl	ap	esslli	battig	up	mcrae	an	ansyn	ansem
best setup on each task														
cnt	74	62	70	59	72	76	66	84	98	41	27	49	43	60
pre	84	75	**80**	**70**	**80**	91	75	86	**99**	41	28	**68**	**71**	**66**
best setup across tasks														
cnt	70	62	70	57	72	76	64	84	98	37	27	43	41	44
pre	83	73	78	68	**80**	86	71	77	98	41	26	67	69	64
worst setup across tasks														
cnt	11	16	23	4	21	49	24	43	38	-6	-10	1	0	1
pre	74	60	73	48	68	71	65	82	88	33	20	27	40	10